TI205 - Introduction à la complexité

Travaux pratiques 2 : Tri fusion

Indications:

- Vous devrez écrire votre code en langage C. Vous pouvez utiliser un éditeur de texte de votre choix pour écrire votre code.
- Ayez une feuille de papier pour répondre aux questions théoriques.
- Le travail est à faire de manière individuelle.
- **Le TP est à rendre sur Moodle avant la fin de la séance sur l'espace dédié de dépôt**.
- Rendre dans un unique fichier `.zip` l'ensemble des fichiers nécessaires à la compilation et l'exécution de votre programme. Ainsi que les fichiers `merge-sort-complexity.txt` et `execution-trace.txt`.
- Pour les questions portant sur la complexité de vos fonctions, indiquez en commentaire au dessus de la définition de la fonction sa complexité.
- **Seules les questions portant sur la complexité sont à répondre dans le code source.**

Construction

On rappelle que l'algorithme de tri fusion, MERGE_SORT(T, l, r), est un algorithme de tri qui utilise l'approche diviser pour régner. Il divise le tableau T en deux sous-tableaux de tailles *égales*, trie récursivement les deux sous-tableaux, puis fusionne les deux sous-tableaux triés en un seul tableau trié. L'algorithme tel que présenté en cours est le suivant:

```
MERGE_SORT(T, l, r)
  if l < r then
      mid ← (r−l)/2 + l
      MERGE_SORT(T, l, mid)
      MERGE_SORT(T, mid + 1, r)
      MERGE(T, l, mid, r)
  end if
```

Implémenter la fonction MERGE_SORT est trivial, la difficulté réside dans l'implémentation de la fonction MERGE. Nous allons donc commencer par écrire la fonction MERGE de manière indépendante.

A) Premier essai pour la fonction merge

La fonction MERGE fusionne deux tableaux trié en un seul. Nous allons tout d'abord l'écrire d'une manière qui n'est pas nécessairement adaptée à notre algorithme MERGE_SORT. On considère alors deux tableaux triés A et B de taille respective n et m.

1. Pour fusionner deux tableaux A et B, on peut remplir un tableau C qui contiendra les éléments de A et B triés. Quelle est la taille du tableau C?

2. L'idée est alors de parcourir en même temps les tableaux A et B en comparant les éléments de A et B deux à deux. On ajoute alors le plus petit élément à C. En indexant respectivement les éléments de A et B par i et j, puis les éléments de C par k, on peut avoir le pseudo-code suivant:

```
k ← 0; i ← 0; j ← 0
while k < n + m do
  if i < n and j < m then
      if A[i] < B[j] then
          C[k] ← A[i]
          i ← i + 1
      else
          C[k] ← B[j]
          j ← j + 1
      k ← k + 1
  // Sinon si on a fini de parcourir B, ajouter les éléments restants de A
  ...
  // Sinon si on a fini de parcourir A, ajouter les éléments restants de B
  ...
```

Les cas en dehors de $i < n$ and $j < m$ indiquent que l'on a fini de parcourir un des deux tableaux (c'est dû au fait que soit $n < m$ ou que $m > n$). Complétez le pseudo-code ci-dessus ajouter les éléments restants.

3. Écrire une fonction MERGE_AUX en C, qui prend en paramètre deux tableaux triés A et B de tailles n et m respectivement. Cette fonction dois allouer puis renvoyer un tableau C trié contenant les éléments de A et B.

   Le prototype de la fonction est le suivant:

```c
int * merge_aux(int * a, int n, int * b, int m);
```

4. Testez ensuite votre fonction dans un programme principal. Par exemple, pour les tableaux $A = \{1, 3, 5, 7\}$ et $B = \{2, 4, 6, 8\}$, vous devriez obtenir le tableau $C = \{1, 2, 3, 4, 5, 6, 7, 8\}$.

5. Pour cette question uniquement, ajouter une ligne qui affiche les valeurs des variables i, j et k avant de retourner le tableau C. Quelle est la complexité de la fonction MERGE_AUX?

B) Fonction merge pour merge_sort

Une fois le principe de fusion compris, nous allons écrire une fonction de fusion dont le prototype est le suivant:

```c
void merge(int * t, int l, int mid, int r);
```

Ici nous avons un seul tableau T et deux sous-tableaux $T[l, \ldots, mid]$ et $T[mid + 1, \ldots, r]$ qui sont triés. La fonction MERGE doit fusionner les deux sous-tableaux en un seul tableau trié. Pour cela, on peut utiliser une approche similaire à celle de la question précédente.

En utilisant la même approche que précédemment, il nous faut alors remplir un tableau C avec les éléments des deux sous-tableaux.

1. Quelle est la taille du tableau C en fonction de l et de r?

2. Si on indexe respectivement par i et par j les éléments de $T[l, \ldots, mid]$ et de $T[mid + 1, \ldots, r]$; puis par k les éléments de C. Adapter la fonction MERGE_AUX pour fusionner dans C les deux sous-tableaux.

3. Après la fusion dans C, puisque la fonction MERGE doit réarranger les éléments de T entre les indices l et r, il est reste à copier les éléments de C dans T. On a alors:

```
for i from l to r do
    T[i] ← C[k]
```

Exprimer k en fonction de i de l pour que la boucle ci-dessus puisse copier les éléments de C dans T.

4. Écrire alors finalement le code de la fonction MERGE en C. Quelle est la complexité de la fonction MERGE?

5. Testez votre fonction MERGE dans un programme principal. Par exemple, pour le tableau $T = \{1,3,5,7,2,4,6,8\}$, après l'appel `merge(T, 0, 3, 7)`, vous devriez obtenir le tableau $T = \{1,2,3,4,5,6,7,8\}$.

C) Fonction merge_sort

Nous avons maintenant toutes les briques nécessaires pour écrire la fonction MERGE_SORT.

1. Écrire alors la fonction MERGE_SORT en C. Le prototype de la fonction est le suivant:

```
void merge_sort(int * t, int l, int r);
```

2. Testez votre fonction MERGE_SORT dans un programme principal. Par exemple, pour le tableau

$$T = \{8,7,6,5,4,3,2,1\},$$

l'appel `merge_sort(T, 0, 7)` doit modifier le tableau T en $T = \{1,2,3,4,5,6,7,8\}$.

Complexité

Nous allons par la suite analyser la complexité de l'algorithme de tri fusion. Cette complexité a été étudiée en cours, et nous avons conclu que la complexité de l'algorithme de tri fusion est en $O(n \log n)$. L'opération qui contribue le plus à la complexité de la fonction MERGE_SORT se situe dans la fonction MERGE. En particulier la comparaison $t[i] < t[j]$. Nous allons compter le nombre de comparaisons effectuées par la fonction MERGE pour un tableau de taille n.

1. Reprendre le compteur d'appels récursifs `counter_rec` et le compteur d'itérations `counter_iter` que vous avez écrits dans le TP précédent. Ainsi que les fonctions `increment_counter` et `reset_counter`.
   a) Modifier la fonction MERGE pour qu'elle incrémente `counter_iter` à chaque comparaison `t[i]<t[j]`.
   b) Modifier la fonction MERGE_SORT pour qu'elle incrémente `counter_rec` à chaque appel récursif.

2. Écrivez ensuite les fonctions suivantes:
   a) `int * new_array(int n)` qui alloue et retourne un tableau de taille n.
   b) `void delete_array(int * a)` qui libère la mémoire allouée pour un tableau.
   c) `void build_array(int * a, int n)` qui remplit le tableau a avec les valeurs $\{n, n-1, \ldots, 1\}$.

3. Remplissez le programme principal suivant:
   - Initialisez un tableau `sizes` avec les 15 premières puissances de 2.
   - Pour chaque taille n dans `sizes`, créez un tableau a de taille n, remplissez-le avec les valeurs $\{n, n-1, \ldots, 1\}$, puis triez-le avec MERGE_SORT. Affichez ensuite les valeurs n, `counter_iter`, `counter_rec` et $n \log n$.

     Une fonction `log2` peut être directement importée de la bibliothèque `math.h`.
   - Réinitialisez les compteurs `counter_iter` et `counter_rec` après l'affichage des résultats, et avant l'itération suivante.

```
int main () {
    /** COMPLEXITE **/

    // Initialise un tableau sizes avec les 15 premières puissances de 2
    int * sizes = new_array(15);
    sizes[0] = 1;
    for (int i = 1; i < 15; ++i) {
        sizes[i] = 2 * sizes[i-1];
    }

    printf("n \t #calls \t #comp. \t nlogn\n");

    for (int i = 0; i < 15; ++i) {
        int n = sizes[i];
        int * a = new_array(n);

        // Remplir a avec les valeurs {n, n-1, ..., 1}
        build_array(a, n);

        // Trier a avec merge_sort
        // ...

        // Afficher les résultats
        // n, nombres d'appels, nombres de comparaisons, nlogn
        // ...

        // Réinitialiser les compteurs
        // ...
        // ...

        delete_array(a);
    }

    delete_array(sizes);
    return 0;
}
```

Un exemple de sortie attendue est le suivant:

```
     n          #calls          #comp.              nlogn
     1               1               0               0.00
     2               3               1               2.00
     4               7               4               8.00
     8              15              12              24.00
    16              31              32              64.00
    32              63              80             160.00
    64             127             192             384.00
   128             255             448             896.00
   256             511            1024            2048.00
   512            1023            2304            4608.00
  1024            2047            5120           10240.00
  2048            4095           11264           22528.00
  4096            8191           24576           49152.00
  8192           16383           53248          106496.00
 16384           32767          115688          229376.00
```

4. Trouver une constante c telle que le nombre de comparaisons effectuées par la fonction MERGE est $c \cdot n \log n$? Modifiez le programme principal pour ajouter une colonne d'affichage afin de faire appararaître $c \cdot n \log n$.

5. Copiez ensuite la sortie de votre programme dans un fichier `merge-sort-complexity.txt`. **Ce fichier sera à rendre avec le code source**.

Trace d'exécution

Nous allons maintenant ajouter des affichages dans le code de la fonction MERGE et de la fonction MERGE_SORT. Ces affichages doivent permettre de suivre l'exécution de l'algorithme de tri fusion.

Modifier les fonctions MERGE et MERGE_SORT afin qu'elles produisent les affichages suivants:

1. Pour la fonction MERGE_SORT: un affichage `"Sorting [l, r]"` avant de trier les éléments de $T[l,\ldots,r]$ et `"Sorted [l, r]"` quand $T[l,\ldots,r]$ est trié.

2. Pour la fonction MERGE: un affichage `"Merging [l, mid] and [mid + 1, r]"` avant la fusion de $T[l,\ldots,mid]$ et $T[mid+1,\ldots,r]$ et `"Merged [l, mid] and [mid + 1, r]"` après la fusion, suivi de l'affichage des éléments fusionnés.

3. Testez votre programme avec **un tableau T initialisé avec les chiffres composant votre numéro d'étudiant**. Par exemple, si votre numéro d'étudiant est 20230778, vous devez initialiser T à $T = \{2,0,2,3,0,7,7,8\}$.

4. Copier la sortie de votre programme dans un fichier `execution-trace.txt`. **Ce fichier sera à rendre avec le code source**.

Un exemple d'affichage attendu pour le tableau $T = \{2, 0, 2, 3, 0, 7, 7, 8\}$ est le suivant:

```
Original array : 2 0 2 3 0 7 7 8
Sorting [0, 7]
Sorting [0, 3]
Sorting [0, 1]
Sorting [0, 0]
Sorted  [0, 0]
Sorting [1, 1]
Sorted  [1, 1]
        Merging [0, 0] and [1, 1]
        Merged  [0, 0] and [1, 1] into {0 2}
Sorted  [0, 1]
Sorting [2, 3]
Sorting [2, 2]
Sorted  [2, 2]
Sorting [3, 3]
Sorted  [3, 3]
        Merging [2, 2] and [3, 3]
        Merged  [2, 2] and [3, 3] into {2 3}
Sorted  [2, 3]
        Merging [0, 1] and [2, 3]
        Merged  [0, 1] and [2, 3] into {0 2 2 3}
Sorted  [0, 3]
Sorting [4, 7]
Sorting [4, 5]
Sorting [4, 4]
Sorted  [4, 4]
Sorting [5, 5]
Sorted  [5, 5]
        Merging [4, 4] and [5, 5]
        Merged  [4, 4] and [5, 5] into {0 7}
Sorted  [4, 5]
Sorting [6, 7]
Sorting [6, 6]
Sorted  [6, 6]
Sorting [7, 7]
Sorted  [7, 7]
        Merging [6, 6] and [7, 7]
        Merged  [6, 6] and [7, 7] into {7 8}
Sorted  [6, 7]
        Merging [4, 5] and [6, 7]
        Merged  [4, 5] and [6, 7] into {0 7 7 8}
Sorted  [4, 7]
        Merging [0, 3] and [4, 7]
        Merged  [0, 3] and [4, 7] into {0 0 2 2 3 7 7 8}
Sorted  [0, 7]
Sorted array : 0 0 2 2 3 7 7 8
```